

Consumo de API com HTML e JavaScript

Uma introdução ao `fetch` e `axios`

O que é uma API?

API significa **A**pplication **P**rogramming **I**nterface (Interface de Programação de Aplicações).

É um conjunto de regras e definições que permite que diferentes aplicações se comuniquem entre si.

Funciona como um "contrato" que define como um software pode interagir com outro.

API REST

REST (Representational State Transfer) é um estilo de arquitetura para criar APIs. As APIs que seguem o padrão REST são chamadas de **RESTful**.

- **Cliente-Servidor:** Separação de responsabilidades.
- **Stateless:** Cada requisição é independente.
- **Uso de métodos HTTP:** GET , POST , PUT , DELETE .
- **Uso de URLs (Endpoints):** Para identificar os recursos (ex: /character).

O que é uma Promise?

Uma **Promise** é um objeto que representa o resultado futuro de uma operação assíncrona.

Ela pode estar em 3 estados:

- 🕒 **Pending** — operação em andamento
- ✅ **Fulfilled** — operação concluída com sucesso
- ❌ **Rejected** — operação falhou

```
const promessa = new Promise((resolve, reject) => {  
  const sucesso = true;  
  if (sucesso) resolve('Deu certo!');  
  else reject('Algo deu errado');  
});  
  
promessa.then(resultado => console.log(resultado));
```

A API do Rick and Morty

Para nossos exemplos, usaremos a [The Rick and Morty API](#).

- **URL Base:** `https://rickandmortyapi.com/api`
- **Endpoints Principais:**
 - `/character` : Para personagens.
 - `/location` : Para localizações.
 - `/episode` : Para episódios.

Exemplo: Para buscar o personagem com ID 2 (Morty Smith), usamos a URL:

`https://rickandmortyapi.com/api/character/2`

Método 1: `fetch` API

O `fetch` é uma API nativa dos navegadores modernos para fazer requisições de rede. Ele é baseado em *Promises*.

Sintaxe com `.then()`

```
fetch('https://rickandmortyapi.com/api/character/1')
  .then(response => {
    if (!response.ok) {
      throw new Error(`Erro HTTP: ${response.status}`);
    }
    return response.json();
  })
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.error('Erro:', error);
  });
```

fetch com async/await

async/await é uma forma mais "limpa" e moderna de trabalhar com *Promises*.

```
async function getCharacter() {  
  try {  
    const response = await fetch('https://rickandmortyapi.com/api/character/1');  
    if (!response.ok) {  
      throw new Error(`Erro HTTP: ${response.status}`);  
    }  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.error('Erro:', error);  
  }  
}  
  
getCharacter();
```

Exemplo Prático com `fetch`

Vamos buscar um personagem e exibi-lo na tela.

HTML:

```
<h1>Personagem de Rick and Morty</h1>  
<div id="character-container"></div>
```

JavaScript:

```
const container = document.getElementById('character-container');

async function fetchCharacter() {
  try {
    const response = await fetch('https://rickandmortyapi.com/api/character/1');
    if (!response.ok) {
      throw new Error(`Erro HTTP: ${response.status}`);
    }
    const data = await response.json();

    container.innerHTML = `
      <h2>${data.name}</h2>
      
      <p>Status: ${data.status}</p>
      <p>Espécie: ${data.species}</p>
    `;
  } catch (error) {
    container.innerHTML = `<p>Erro ao buscar personagem: ${error.message}</p>`;
  }
}

fetchCharacter();
```

Método 2: `axios`

`axios` é uma biblioteca de cliente HTTP baseada em *Promises* para navegador e Node.js. Ela simplifica o processo de fazer requisições.

Vantagens:

- Sintaxe mais simples.
- Converte dados para JSON automaticamente.
- Melhor tratamento de erros.

Instalação (via CDN):

```
<script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
```

axios com .then()

```
axios.get('https://rickandmortyapi.com/api/character/2')
  .then(response => {
    // A resposta já vem em JSON no objeto 'data'
    console.log(response.data);
  })
  .catch(error => {
    console.error('Erro:', error);
  });
```

axios com async/await

```
async function getCharacter() {  
  try {  
    const response = await axios.get('https://rickandmortyapi.com/api/character/2');  
    console.log(response.data);  
  } catch (error) {  
    console.error('Erro:', error);  
  }  
}  
  
getCharacter();
```

Enviando Dados com POST

Além de buscar dados (GET), precisamos **enviar dados** para a API (POST).

Com `fetch`

```
async function criarPersonagem() {  
  const response = await fetch('https://jsonplaceholder.typicode.com/posts', {  
    method: 'POST',  
    headers: { 'Content-Type': 'application/json' },  
    body: JSON.stringify({  
      title: 'Novo Personagem',  
      body: 'Rick Sanchez',  
      userId: 1  
    })  
  });  
  const data = await response.json();  
  console.log(data);  
}
```

Com `axios`

```
async function criarPersonagem() {  
  const response = await axios.post('https://jsonplaceholder.typicode.com/posts', {  
    title: 'Novo Personagem',  
    body: 'Rick Sanchez',  
    userId: 1  
  });  
  console.log(response.data);  
}
```

Note como `axios` é mais conciso: não precisa de `headers` nem `JSON.stringify`.

fetch vs. axios

Característica	fetch	axios
Nativo	✓ Sim	✗ Não (precisa instalar)
Conversão JSON	Manual (<code>response.json()</code>)	✓ Automática
Tratamento de Erro	Precisa checar <code>response.ok</code> manualmente	✓ Rejeita automaticamente em erros HTTP (404, 500)
Sintaxe	Mais verbosa	Mais concisa
Compatibilidade	Navegadores modernos	Ampla (navegadores e Node.js)

Exemplo Prático: Listando Personagens

Vamos buscar vários personagens e criar uma lista.

HTML:

```
<h1>Personagens de Rick and Morty</h1>  
<ul id="character-list"></ul>  
<script src="https://cdn.jsdelivr.net/npm/axios/dist/axios.min.js"></script>
```

JavaScript (com `axios`):

```
const list = document.getElementById('character-list');

async function fetchAllCharacters() {
  try {
    const response = await axios.get('https://rickandmortyapi.com/api/character');
    const characters = response.data.results;

    characters.forEach(char => {
      list.innerHTML += `<li> ${char.name}</li>`;
    });
  } catch (error) {
    list.innerHTML = `<li>Erro ao buscar personagens: ${error.message}</li>`;
  }
}

fetchAllCharacters();
```

! Erro Comum: CORS

CORS (Cross-Origin Resource Sharing) é um mecanismo de segurança do navegador que bloqueia requisições de um domínio para outro.

Exemplo do erro:

```
Access to fetch at 'https://api.exemplo.com' from origin  
'http://localhost:3000' has been blocked by CORS policy
```

Como resolver:

- O **servidor** precisa enviar o header `Access-Control-Allow-Origin`
- Durante desenvolvimento, use extensões do navegador ou proxies
- Não é possível resolver apenas no front-end!



Exercício Prático

Consuma a [JSONPlaceholder API](#) (API falsa para testes):

1. **GET** — Liste os 5 primeiros posts (`/posts?_limit=5`) e exiba título e corpo na tela
2. **POST** — Crie um formulário HTML que envie um novo post via `axios.post`
3. **Desafio** — Adicione um indicador de "*Carregando...*" enquanto a requisição é feita
4. **Bônus** — Trate o erro e exiba uma mensagem amigável ao usuário
5. entregar no [forms \(clique aqui\)](#)



Dica: Use `async/await` e `try/catch` em todos os exemplos.

Dúvidas?